

INSTITUTO TECNOLÓGICO DE CULIACAN

INGENIERIA EN SISTEMAS COMPUTACIONALES

Proyecto de Inteligencia Artificial: Optimización de Riego con  
Enjambre de Partículas

TOPICOS DE INTELIGENCIA ARTIFICIAL

DENZEL ARTURO GUTIERREZ CHAVEZ

JUAN CARLOS QUIÑONEZ MADRID



TECNOLOGICO  
NACIONAL DE MÉXICO



CULIACAN, SINALOA

08/05/26

Link repo: <https://github.com/CarlosMadrid11/Topicos-de-la-IA> LEER  
EL README DEL MODULO 3 PARA LA EJECUCION DEL NB

## DESCRIPCION DEL PROBLEMA

En el presente proyecto que trabajamos en la materia de Topicos de Inteligencia Artificial, nos enfocamos en una problemática que afecta al sector agrícola, especialmente en al región de Guasave, Sinaloa. Esta problemática se obtiene mediante la carencia de optimización de riego en áreas de cultivo mediante la ubicación estratégica de sensores. La administración eficiente del agua aquí representa un elemento fundamental a la hora de continuar con la producción agrícola, sobre todo en regiones donde las condiciones ambientales y las características del terreno generan variaciones en la retención de humedad y en la eficiencia del riego. Esto cobra mayor relevancia al considerar que la agricultura es el principal consumidor de agua dulce a nivel mundial, utilizando cerca del 70% del total disponible (Zhang et al., 2024).

El objetivo principal consiste en definir la distribución más adecuada de sensores dentro de una zona de cultivo, tomando en cuenta factores como la humedad del suelo, la elevación, la salinidad, la temperatura y el tipo de cultivo. Con ello, se busca mejorar la eficiencia del riego y facilitar una mejor toma de decisiones en la gestión hídrica.

En este contexto, la región de Guasave presenta características particulares, ya que cuenta con una llanura costera con altitudes que oscilan entre los 10 y 50 metros sobre el nivel del mar, además de poseer suelos fértiles con presencia de zonas afectadas por salinidad y erosión superficial. Asimismo, la producción agrícola de la región incluye principalmente cultivos de maíz, tomate y chile. Estas variaciones en las condiciones del terreno y en los tipos de cultivo impactan directamente en los niveles de humedad, haciendo necesario el uso de métodos de optimización inteligentes que permitan adaptar la ubicación de los sensores a las necesidades específicas de cada área.

Por esta razón, se propone resolver el problema de distribución óptima de sensores mediante el uso del algoritmo bioinspirado de Enjambre de Partículas (PSO), con la finalidad de identificar las posiciones que permitan maximizar la eficiencia del sistema de riego.

Conociendo esto podemos emplear el algoritmo de forma eficiente y preparado para generar nuevas alternativas a partir de parámetros que uno desee probar y generar al conseguir resultados efectivos.

## JUSTIFICACION DEL USO DE PSO

Como bien sabemos, el algoritmo PSO fue propuesto por el ingeniero eléctrico Russell C. Eberhart y el psicólogo social James Kennedy. Está inspirado en el comportamiento social y biológico de bandadas de aves buscando fuentes de alimento los individuos se denominan partículas y recorren el espacio de búsqueda buscando la mejor posición global.

Bajo esta explicación, el uso del algoritmo PSO resulta adecuado para resolver el problema de distribución óptima de sensores en campos agrícolas, ya que permite explorar diferentes posiciones dentro del terreno y encontrar aquellas que ofrecen mejores resultados en términos de eficiencia de riego. Cada partícula representa una posible ubicación de los sensores y, a medida que avanza el proceso, estas partículas ajustan su posición considerando tanto su mejor experiencia individual como la mejor solución encontrada por el conjunto.

Además, PSO destaca por su capacidad para trabajar en problemas complejos de optimización continua, especialmente en escenarios donde intervienen múltiples variables, como humedad del suelo, salinidad, temperatura, elevación y tipo de cultivo. Gracias a su mecanismo de búsqueda colaborativa, el algoritmo puede adaptarse a las variaciones presentes en el terreno agrícola y encontrar configuraciones más eficientes en comparación con métodos tradicionales.

En el contexto que se maneja en el presente proyecto, podemos agregar que según expertos:

- PSO tiene una implementación sencilla y un bajo costo computacional en comparación con otros algoritmos metaheurísticos, lo que facilita su aplicación en áreas agrícolas extensas. (**Fuente:** Poli, R., Kennedy, J., & Blackwell, T. , 2007)
- Es ampliamente utilizado en agricultura de precisión y sistemas de riego inteligente para optimizar el uso del agua y mejorar la toma de decisiones agrícolas. (**Fuente:** Alghamdi, 2023).
- El algoritmo PSO es adecuado porque puede resolver problemas de optimización complejos y multidimensionales, como la distribución de sensores considerando humedad, salinidad, temperatura y elevación del terreno. (**Fuente:** Zhang,2024)

## DISEÑO DEL ALGORITMO

El proyecto fue desarrollado en Python por ser el lenguaje estándar en ciencia de datos e inteligencia artificial. Su sintaxis clara lo hace ideal para implementar algoritmos bio-inspirados sin perder legibilidad, y cuenta con un ecosistema de librerías científicas que ningún otro lenguaje iguala en accesibilidad.

En esto también debemos de darle un reconocimiento a las librerías que nos ayudaron a generar y entender el diseño del algoritmos.

- **Pandas** fue la herramienta elegida para cargar y manipular el archivo CSV del campo de Guasave. Permite leer datos tabulares en una sola línea de código y acceder a columnas por nombre, lo cual simplifica enormemente el preprocesamiento.
- **NumPy** complementa a Pandas en todo lo que involucra cálculo numérico: operaciones sobre arreglos, generación de números aleatorios y álgebra lineal. En PSO específicamente, cada posición y velocidad de partícula es un arreglo NumPy, lo que permite actualizar los 45 valores de una partícula en una sola operación vectorizada en lugar de recorrerlos uno por uno.
- **Scikit-learn** aportó el componente de preprocesamiento a través de su clase `MinMaxScaler`. Dado que el dataset contiene columnas con rangos muy distintos — la humedad va de 5 a 45, mientras la latitud varía solo entre 25.52 y 25.62 — el algoritmo PSO calcularía distancias dominadas por las columnas de mayor magnitud. El escalado `MinMax` lleva todas las variables al rango uniforme de 0 a 1, garantizando que cada característica contribuya equitativamente al cálculo de distancias.
- **SciPy** fue utilizada específicamente por su función `cdist`, que calcula en una sola operación matricial la distancia euclidiana entre todos los puntos del campo y todos los sensores propuestos simultáneamente, lo que es considerablemente más eficiente que hacerlo con ciclos anidados.
- **PySwarms** es la librería central del proyecto. Implementa el algoritmo `GlobalBestPSO`, que corresponde a la topología de enjambre donde todas las partículas comparten el mismo mejor global. Esto elimina la necesidad de programar desde cero el ciclo de iteraciones, la actualización de velocidades y el seguimiento del óptimo global, permitiendo enfocarse en definir correctamente la función de costo del problema específico.
- **Matplotlib** y **Seaborn** se usaron para la visualización de resultados: `Matplotlib` para las gráficas de convergencia y `Seaborn` para el mapa de distribución de sensores sobre el campo, aprovechando su integración nativa con `DataFrames` de `Pandas`.

El diseño del algoritmo se estructuró en tres clases con responsabilidades claramente separadas. La clase `CargadorDatos` encapsula el pipeline de preparación de datos mediante el método `cargar_y_preprocesar()`, que ejecuta en secuencia la lectura del CSV, la codificación One-Hot de la columna categórica Cultivo y el escalado MinMax de todas las variables numéricas. La codificación One-Hot es necesaria porque PSO opera exclusivamente sobre números y no puede interpretar texto directamente.

El constructor de la clase `OptimizadorSensoresPSO` recibe todos los parámetros necesarios para configurar el problema de optimización y calcula las dimensiones del espacio de búsqueda. El número total de dimensiones se define como el producto del número de sensores por el número de características, ya que cada sensor debe posicionarse en el espacio multidimensional de características.

```
class OptimizadorSensoresPSO:

    def __init__(self, n_sensores, datos, n_particulas, iteraciones,
                  escalador, columnas_caracteristicas, opciones_pso):

        self.n_sensores = n_sensores
        self.datos = datos
        self.n_particulas = n_particulas
        self.iteraciones = iteraciones
        self.escalador = escalador
        self.columnas_caracteristicas = columnas_caracteristicas
        self.opciones_pso = opciones_pso
        self.n_caracteristicas = datos.shape[1]
        self.n_dimensiones = n_sensores * self.n_caracteristicas

        # Resultados de la optimización
        self.mejor_costo = np.inf
        self.mejor_posicion = None
        self.historial_costos = []
```

El método `_funcion_costo_particula()` recibe el vector de posición de una partícula 45 números representando 5 sensores con 9 características cada uno lo reorganiza en una matriz de 5 filas y calcula la suma de distancias al cuadrado desde cada punto del campo hasta su sensor más cercano. Esta métrica, conocida como inercia total, es la misma que usa el algoritmo k-means y mide qué tan bien los sensores centralizan la cobertura del campo

```
def _funcion_costo_particula(self, posicion_particula):
    try:
        centroides = posicion_particula.reshape(
            self.n_sensores, self.n_caracteristicas
        )
    except ValueError as e:
        print(f" Error de dimensiones: {e}")
        return np.inf

    # descubrimo que cdist calcula TODAS las distancias entre puntos y
    # sensores de golpe
    distancias = cdist(self.datos, centroides, 'euclidean')
    min_distancias = np.min(distancias, axis=1)
    costo = np.sum(min_distancias ** 2)
    return costo
```

El método `_funcion_costo_lote()` aplica esta evaluación a todo el enjambre simultáneamente, ya que PySwarms entrega en cada iteración la posición de todas las partículas en una sola matriz.

```
def _funcion_costo_lote(self, lote_particulas):
    return np.array([
        self._funcion_costo_particula(p) for p in lote_particulas
    ])
```

El método `ejecutar_optimizacion()` configura los límites del espacio de búsqueda entre 0 y 1, instancia el optimizador de PySwarms con los parámetros `c1`, `c2` y `w`, y delega la ejecución del enjambre completo.

```
def ejecutar_optimizacion(self):
    """
    Configura y ejecuta el algoritmo PSO usando pyswarms.
    """
    print(f" Partículas: {self.n_particulas} | "
          f"Iteraciones: {self.iteraciones} | "
          f"Parámetros: {self.opciones_pso}")

    # Límites: todas las dimensiones entre 0 y 1
    limite_inferior = np.zeros(self.n_dimensiones)
    limite_superior = np.ones(self.n_dimensiones)
    limites = (limite_inferior, limite_superior)

    optimizador = ps.single.GlobalBestPSO(
```

```

        n_particles = self.n_particulas,
        dimensions  = self.n_dimensiones,
        options     = self.opciones_pso,
        bounds      = limites
    )

    # aqui se puede notar que optimize() llama a _funcion_costo_lote en
    # cada iteración
    self.mejor_costo, self.mejor_posicion = optimizador.optimize(
        self._funcion_costo_lote,
        iters    = self.iteraciones,
        verbose  = True
    )

    self.historial_costos = optimizador.cost_history

```

Finalmente, `obtener_ubicaciones_optimas()` toma la mejor posición encontrada, invierte el escalado MinMax para recuperar las coordenadas GPS reales y devuelve un DataFrame legible con la ubicación de cada sensor.

```

def obtener_ubicaciones_optimas(self):
    """
    Aqui lo que esperamos es obtener un DataFrame con las coordenadas
    reales de los N sensores en escala
    """
    if self.mejor_posicion is None:
        raise Exception("Debes ejecutar ejecutar_optimizacion() primero.")

    # Reshape: vector plano → N_sensores filas
    ubicaciones_escaladas = self.mejor_posicion.reshape(
        self.n_sensores, self.n_caracteristicas
    )

    # Invertimos el escalado para recuperar valores reales
    # inverse_transform hace la operación inversa al MinMaxScaler
    ubicaciones_originales =
self.escalador.inverse_transform(ubicaciones_escaladas)

    # Creamos un DataFrame legible con nombres de columnas
    df_resultado = pd.DataFrame(
        ubicaciones_originales,
        columns=self.columnas_caracteristicas
    ).round(4)

    # Añadimos etiqueta de sensor

```

```
df_resultado.insert(0, 'Sensor', [f'S{i+1}' for i in
range(self.n_sensores)])
return df_resultado
```

## DISEÑO DEL ALGORITMO

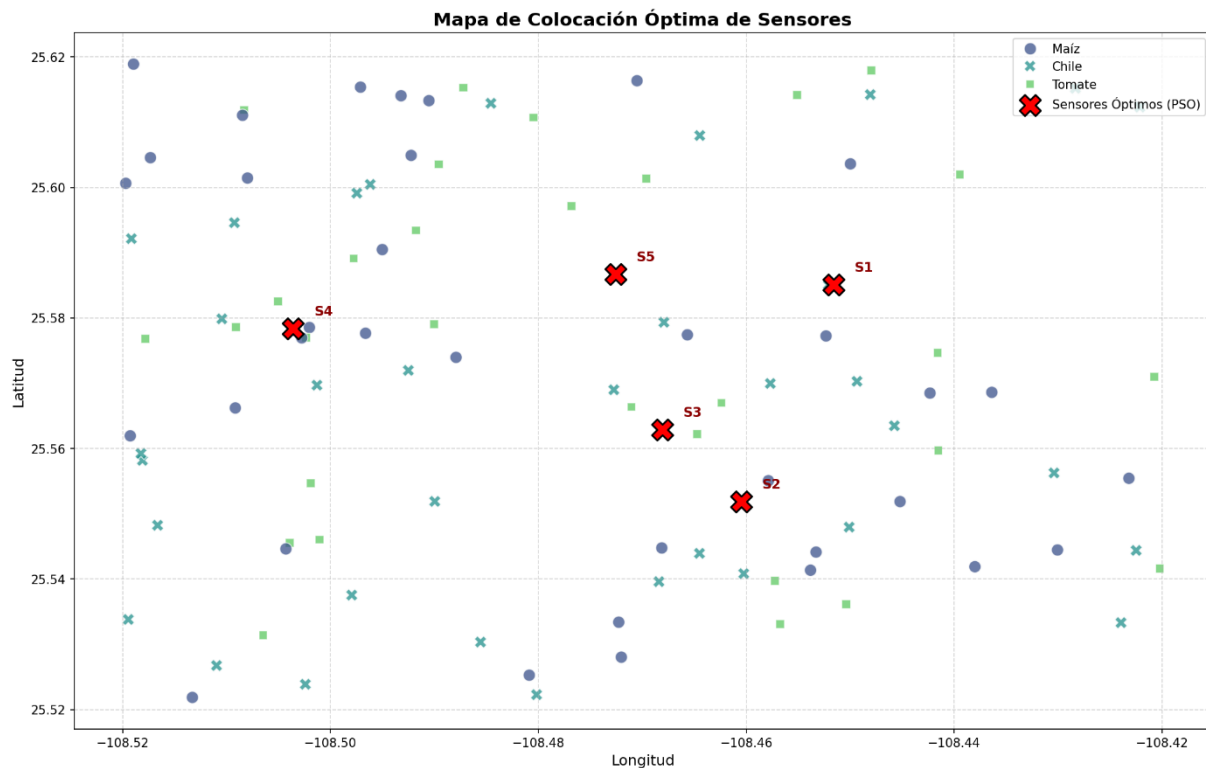
El algoritmo de Optimización por Enjambre de Partículas fue ejecutado con una configuración de 50 partículas durante 100 iteraciones, con el objetivo de determinar la ubicación óptima de 5 sensores de humedad en el campo agrícola de Guasave, Sinaloa. Al finalizar el proceso de optimización, el algoritmo alcanzó un costo mínimo de 71.1473, valor que representa la suma de distancias al cuadrado entre cada punto del campo y su sensor más cercano, siendo este el criterio de optimización minimizado por el enjambre.

La siguiente tabla presenta las coordenadas GPS y las condiciones ambientales estimadas para cada sensor en su posición óptima:

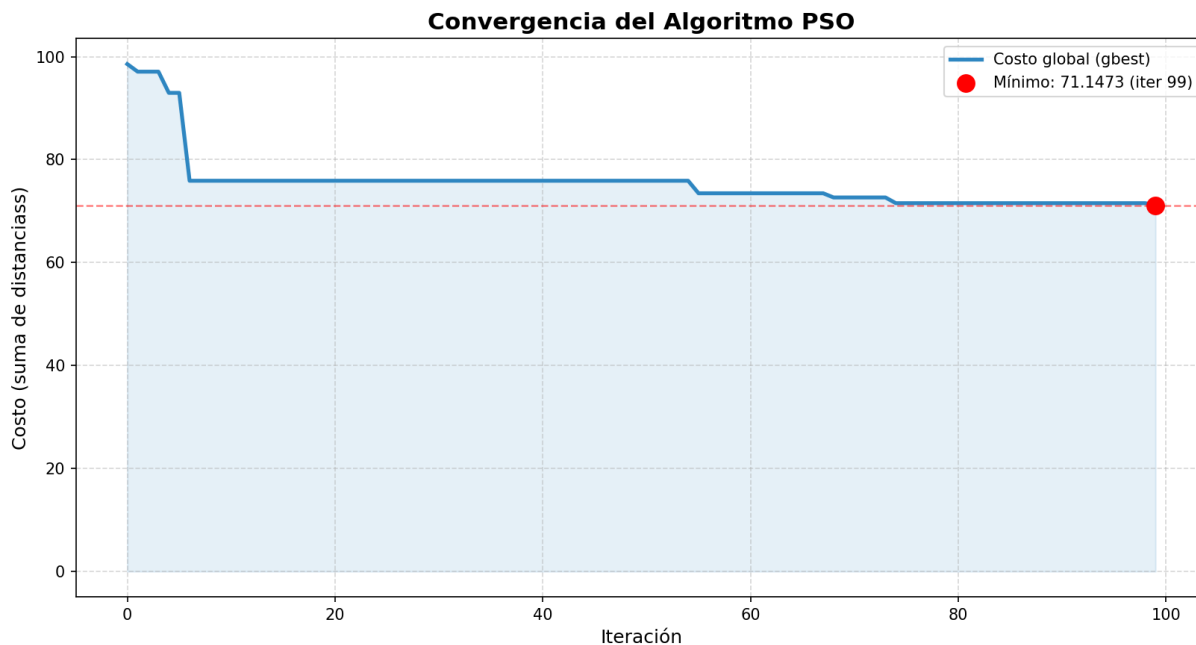
| Sensor | Latitud | Longitud  | Humedad (%) | Salinidad (dS/m) | Temperatura (°C) |
|--------|---------|-----------|-------------|------------------|------------------|
| S1     | 25.5851 | -108.4516 | 40.48       | 2.58             | 29.88            |
| S2     | 25.5519 | -108.4605 | 29.27       | 2.08             | 31.76            |
| S3     | 25.5629 | -108.4681 | 18.38       | 1.31             | 30.82            |
| S4     | 25.5784 | -108.5036 | 23.75       | 1.41             | 32.44            |
| S5     | 25.5867 | -108.4726 | 23.90       | 2.18             | 27.07            |



En este apartado mostramos como se ve mapeado la ubicación de los 5 sensores dentro del mapa de colocación



Ademas de este resultado, indagamos por Internet y encontramos la forma de analizar como es que fue moviendo de costo el movimiento por cada Iteracion gracias a el proceso de convergencia del algoritmo, anexando así, la imagen donde se presenta como es que llego a ese resultado decidido.



## ANALISIS DE EFICIENCIA

Nosotros pensamos que el análisis del proyecto fue eficiente debido a que el algoritmo PSO permitió encontrar soluciones óptimas sin requerir procedimientos matemáticos complejos, como el cálculo de derivadas o modelos analíticos avanzados. Gracias a esto, el procesamiento de datos relacionados con humedad, salinidad, temperatura y elevación del terreno pudo realizarse de manera más rápida y con menor consumo de recursos computacionales.

PSO trabaja mediante partículas que exploran simultáneamente diferentes soluciones dentro del espacio de búsqueda, lo que mejora la velocidad de convergencia hacia configuraciones adecuadas para la ubicación de sensores. Esta característica permitió analizar múltiples posibilidades de distribución de sensores en el terreno agrícola sin incrementar excesivamente el tiempo de ejecución

## CONCLUSIONES

Creemos que el presente proyecto cumplió con todo lo que pensamos nosotros que íbamos a conseguir, tuvo como objetivo principal determinar la ubicación óptima de cinco sensores de humedad en un campo agrícola del municipio de Guasave, Sinaloa, mediante la implementación del algoritmo PSO, y con base a los resultados obtenidos podemos concluir que se cumplió satisfactoriamente el propósito del proyecto.

El algoritmo PSO demostró ser una herramienta eficaz para resolver el problema de posicionamiento óptimo de sensores, logrando identificar cinco ubicaciones geográficas distribuidas de forma estratégica sobre el campo de estudio. La solución encontrada alcanzó un costo mínimo de 71.1473, valor que representa la menor suma de distancias al cuadrado entre los cien puntos del campo y sus sensores más cercanos, garantizando que ninguna zona del terreno quedara desatendida de forma significativa.

En términos generales, el proyecto logró integrar de forma coherente el conocimiento sobre algoritmos bio-inspirados, preprocesamiento de datos, programación orientada a objetos en Python y visualización de resultados, demostrando que la inteligencia de enjambre representa una alternativa viable, eficiente y fundamentada para resolver problemas de optimización espacial en contextos agrícolas reales.

## Referencias:

- Freitas, D., Lopes, LG y Morgado-Dias, F. (2020). Optimización por enjambre de partículas: una revisión histórica hasta los desarrollos actuales. *Entropy* , 22 (3), 362. <https://doi.org/10.3390/e22030362>
- Zhang, Y., Wang, S., & Ji, G. (2015). A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications. *Mathematical Problems in Engineering*, 2015, Article ID 931256. <https://doi.org/10.1155/2015/931256>
- Al-sammarraie MAJ, Ilbas AI. 2024. Aprovechamiento de las técnicas de automatización para apoyar la sostenibilidad en la agricultura. *Tecnología en Agronomía* 4: e029doi: [10.48130/tia-0024-0026](https://doi.org/10.48130/tia-0024-0026)
- Poli, Riccardo & Kennedy, James & Blackwell, Tim. (2007). Particle Swarm Optimization: An Overview. *Swarm Intelligence*. 1. 10.1007/s11721-007-0002-0.
- Tian, Y.; Wang, J.; Chen, J.; Yu, D.; Zeng, Z.; Fu, J.; Zhang, F.; Cao, H.; Liu, F.; Liang, T. Effects of Integrated Management Strategies on Pepper Yield and Quality: A Study of Cultivation and Nutrient Management Practices. *Agronomy* 2024, 14(12), 2754; <https://doi.org/10.3390/agronomy14122754>.  
<https://www.mdpi.com/2073-4395/14/12/2754>